

PhantomCortex

Full System Report

A Whole-Codebase Audit of the Five-Signal Malware Detection Pipeline

Covering the entire correction history since baseline commit 9331159
through HEAD 438d936 — all five signals, 40 commits, 10 originally-audited issues

Prepared 2026-09-22 · cortex-ml repository

Table of Contents

1.	Executive Summary
2.	The Problem And The System
3.	How A Decision Gets Made
4.	Where This Started: The Original Issues
5.	The Full Correction Timeline
6.	Issue-by-Issue Resolution Audit
7.	Per-Signal Current State
8.	This Session's Specific Work: The Static Retrain
9.	What's Better Now, System-Wide
10.	What's Still Open, System-Wide
11.	What Changed, Concretely
12.	What's Left

1. Executive Summary

PhantomCortex is a five-signal Windows malware detection pipeline: a static PE classifier, a behavioral API-call-sequence classifier, a memory-forensics classifier, a network-flow classifier, and an experimental process-emulation classifier, combined by a single sequential-priority policy engine rather than an averaged or blended score. Its purpose is to catch malware across several independent angles of evidence — what a file looks like on disk, what it does when it runs, what a memory snapshot shows, what its network traffic looks like, and what an emulator predicts it would do — so that no single signal's blind spot is the whole system's blind spot.

The system did not start from a clean slate. An earlier audit, compiled at commit 9331159, documented ten concrete correctness problems: a probability-calibration bug that compressed scores into a narrow band, a calibrator fit on the same data used for early stopping, decision thresholds chosen by peeking at the test set and then re-used to report results on that same test set, raw AUC numbers that hid the real false-positive burden a low-prevalence deployment would see, a behavioral model at risk of learning dataset shortcuts instead of real behavior, feature-extraction failures that silently became misleading zero vectors, a training/inference feature-extractor pair with no automated agreement check, a broken export-count feature, and two separate ways an analysis failure could be misreported as a malware finding. This was not a single bug list closed in one pass. It has been an ongoing, multi-phase correction effort spanning eight distinct phases of work across four dates (2026-09-08, 2026-09-10, 2026-09-18, and this session's 2026-09-21 static retrain), each phase closing one or more of the original issues or hardening the system against a newly-discovered real-world failure mode.

Reading the current code directly (not the trackers, not the prior report) shows eight of the ten original issues now FIXED, with concrete evidence at file and line level, and two PARTIALLY FIXED: the behavioral model's padding-mask and short-trace problems are fixed, but a narrower dataset-shortcut risk (Cuckoo-sandbox sentinel tokens present only in one malicious-only source dataset) is still unaddressed and not even tracked; and the feature-parity test between training and inference exists as a real, working MVP but explicitly does not yet cover the two hardest comparisons — a real EMBER2024 record-schema check and a skew-quantification cross-check against the reference thrember extractor, both still blocked by a dependency conflict. Static's once-uncapped BLOCK authority is now interim-capped to ALERT pending that same feature-parity work, escalating to a real BLOCK only when a second, independent signal corroborates it. This session added a fourth model retrain — Cortex-Static, following the same split-discipline pattern already proven on Memory and Network — closing the calibration and threshold-leakage issues for the last of the three tree-based signals.

2. The Problem And The System

PhantomCortex analyzes a file (and, optionally, an API-call trace, a memory-forensics feature vector, and a network-flow feature vector supplied alongside it) and produces one of five outcomes: ALLOW, NEEDS_REVIEW, ALERT, BLOCK, or TERMINATE. It uses five independent signals rather than one, because each looks at a different kind of evidence and none of them is trustworthy alone in every situation:

- **Cortex-Static** reads the file's raw bytes (a 2568-dimension EMBER2024-compatible feature vector) and scores it with a LightGBM model — the fastest, cheapest signal, but structurally unable to see runtime behavior, and the one this session retrained.
- **Cortex-Behavioral** reads a supplied sequence of Windows API calls and scores it with a 1D-CNN plus self-attention network — it can catch what a file does even when the file itself looks clean, and it holds the system's highest, uncapped authority (TERMINATE).

- **Cortex-Memory** reads a supplied memory-forensics feature vector (CIC-MalMem-2022-derived) and scores it with LightGBM — built to catch injected or fileless activity that never touches disk, so it runs independently of whether a valid file was even supplied.
- **Cortex-Network** reads a supplied network-flow feature vector (CSE-CIC-IDS2018-derived) and scores it with LightGBM — built to catch command-and-control or exfiltration traffic, and, like Memory, runs unconditionally.
- **Cortex-Emulation** reads a supplied Speakeasy-emulation API-name sequence and scores it with the same 1D-CNN-plus-attention architecture as Behavioral — trained and evaluated, but held at report-only status because of a confirmed concept-drift collapse across collection eras (Part 7 has the evidence).

No two signals' scores are ever averaged or blended. A single policy engine (`inference/policy_engine.py::decide()`) combines the five verdicts through a strict, ordered priority chain, described next.

3. How A Decision Gets Made

Figure 2 (`full_scan_sequence.png`) traces the real, current sequence inside `inference/pipeline.py::CortexPipeline.scan()`, function by function, as it actually reads today. In plain terms:

- Memory and Network run **first**, before any file is even opened — they do not depend on a valid on-disk PE, because their entire purpose is catching threats (injected code, C2 traffic) that may have no file to look at.
- The file is validated (does it exist, is it a readable file under 100 MiB, is it structurally a Windows PE). Any failure here means the scan could not be completed — not that something suspicious was found.
- A known-file allowlist check runs next: an NSRL SHA-256 hash match, or a real Authenticode signature chain verified to a trusted root, marks the file ALLOW immediately and skips the machine-learning step entirely for Static — while Memory, Network, and Behavioral still run on it exactly as they would on any other file.
- If neither allowlist check matched, Cortex-Static extracts its 2568-dimension feature vector and scores it. Any feature group that failed to extract is recorded, not silently replaced with zeros.
- Cortex-Behavioral then runs, if an API-call trace was supplied and Static did not hit a hard extraction error — deliberately including the case where Static scored BLOCK, so a completed Behavioral MALICIOUS finding can still escalate the file past Static's own capped authority.
- If a feature group that materially changes the verdict's meaning (not a group where an all-zero value is itself a normal, legitimate answer) failed to extract, the scan is routed to NEEDS_REVIEW instead of trusting a score built on missing data.
- Finally, the policy engine combines all four verdicts (Static, Behavioral, Memory, Network) through the fixed priority order shown in Figure 1 below into one final decision, and the scan is packaged into a structured security event with a timestamp and a correlation ID.

PhantomCortex — Five-Signal Architecture (current, HEAD 438d936)

Authority levels, split schemes, and the interim static-BLOCK cap

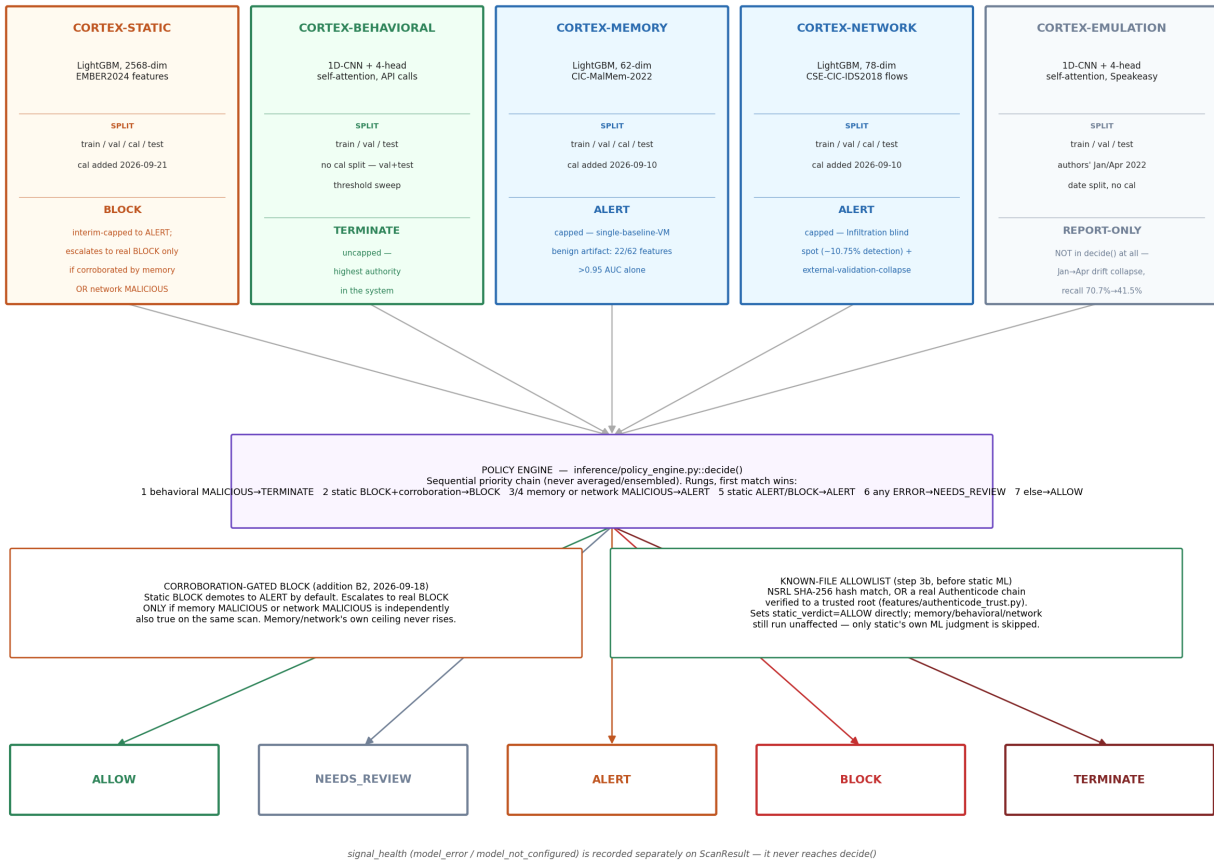


Figure 1 — all five signals, current authority levels, split schemes, and the policy engine's priority chain.

inference/pipeline.py :: CortexPipeline.scan() — real current sequence



Rule of thumb: memory & network are always-on and unconditional; behavioral is gated behind a non-ERROR static verdict;
 only a static ERROR (not ALLOW/ALERT/BLOCK) skips behavioral entirely.

Figure 2 — the real current scan() sequence, function by function (inference/pipeline.py).

Two design choices run through this sequence and matter for everything that follows. First, evidence that a channel *found something* always outranks another channel merely *failing to run* — a completed malicious finding on one signal wins even if a different signal errored out on the same scan. Second, model health (a signal that was never configured, or a configured model that raised an exception at runtime) is recorded separately from the security verdict, so a not-yet-deployed signal cannot force every scan into review, while a genuine runtime failure still routes to a human queue rather than being silently ignored.

4. Where This Started: The Original Issues

An earlier audit, compiled at commit 9331159 (before Memory, Network, and this system's various corrections existed in their current form), found ten concrete problems. Restated in plain terms, before Part 6 shows which are now fixed:

#	In plain terms
1	A scoring model's confidence numbers were computed the wrong way, so real scores got squeezed into a narrow band near the decision line — making it hard to tell a borderline file from a clearly malicious one.
2	The same slice of data used to decide when to stop training a model was then also used to calibrate its confidence numbers — like grading your own homework with the answer key you used to study.
3	Decision thresholds were picked by looking at the exact same test data that was then used to report how well the system performs — the equivalent of choosing a passing grade after seeing the exam answers.
4	The headline accuracy numbers were measured on artificially balanced data, hiding how many false alarms the system would generate in a real deployment where the overwhelming majority of files are harmless.
5	The behavioral model, which watches sequences of program actions, risked learning to recognize which research dataset a sample came from rather than learning what malicious behavior actually looks like.
6	When the system failed to read some property of a file, it silently filled in a default value of zero — which can itself look suspicious (e.g. "no digital signature"), quietly nudging the system toward false alarms without anyone knowing an extraction had failed.
7	The code that turns a file into training data and the code that turns a file into a live prediction were two separate, hand-written implementations with no automatic check that they agree — so a mismatch between them could silently corrupt every live score.
8	A feature meant to count a file's exported functions was, due to a bug, always reporting the same fixed number regardless of the file — a broken input the model was trained to rely on.
9	Unrelated problems — a missing file, a corrupted file, an oversized file, an unavailable model — were all reported as if the system had found something suspicious, when in fact it simply couldn't analyze the file at all.
10	If the memory-analysis model wasn't even running, that missing-model condition itself got reported upward as evidence of malware — a system health problem disguised as a security finding.

Part 6 below re-states these ten issues in their original technical form and gives the current, code-verified status of each.

5. The Full Correction Timeline

`git log 9331159..HEAD --oneline` lists 40 commits. Read individually they are hard to reason about; grouped by what problem each one actually solved, they fall into eight phases. Phases A–E happened in a single hardening session on 2026-09-08; Phase F (the Memory + Network retrain) followed on 2026-09-10; Phase G (the static false-positive severity cluster) on 2026-09-18; and Phase H, this session's static retrain, on 2026-09-21.



Figure 4 — the eight-phase correction timeline.

Phase A — Calibration fix (2026-09-08)

The Platt calibrator on all three LightGBM signals was refit on the raw booster margin instead of the already-sigmoided probability, and thresholds were re-derived against the new calibrated scale.

Closes: Issue 1

Before/after evidence: **Before** (models/static_lgbm.py, commit 6eed759's parent): calibrator fit on `booster.predict(X)` (probability). **After:** fit on `booster.predict(X, raw_score=True)` (margin) — models/static_lgbm.py:63–74, 108–109.

Phase B — Static BLOCK interim cap (2026-09-08)

Static's BLOCK verdict, previously the one uncapped autonomous authority in the whole system, was demoted to ALERT inside `decide()`; Behavioral was also made to run even when Static scored BLOCK, so a completed Behavioral finding can still escalate the file.

Closes: Real-world false-positive risk (not one of the numbered 10, but the direct trigger for issues 2/3's fix)

Before/after evidence: inference/policy_engine.py:432–479 documents the evidence: 2 of 5 confirmed-benign real files still landed at/above the BLOCK threshold post-calibration-fix.

Phase C — Feature + behavioral fixes (2026-09-08)

ExportsInfo was fixed to report the real export count instead of a constant; signify was hard-pinned to 0.9.x with a fail-loud import instead of a silent-zero fallback; the behavioral tokenizer was reconciled so 10–99-call traces are scored the same way at inference as they are in training.

Closes: Issues 5 (short-trace part), 8

Before/after evidence: features/pe_features.py:585–599 (ExportsInfo); tokenizer/api_tokenizer.py:24–30, 74–102 (short-trace floor).

Phase D — Network wiring + test scaffolding (2026-09-08)

Cortex-Network was actually wired into scan() (previously the threshold and verdict function existed but pipeline.py had no network parameter at all); config/thresholds.yaml became the single source of truth for every threshold; the first feature-parity test and policy-engine truth-table tests were added.

Closes: Issue 7 (MVP)

Before/after evidence: inference/pipeline.py:159–164, 369–386 (network wiring); tests/test_static_feature_parity.py (parity MVP).

Phase E — NEEDS_REVIEW + health separation (2026-09-08)

A new NEEDS_REVIEW final-decision state was added so "could not analyze" stopped being reported as "found something"; model health (not configured vs. raised at runtime) was split onto a separate signal_health field instead of feeding the security verdict; feature-degradation was made explicit instead of silent; and realistic-prevalence PPV/alert-rate projections were added to evaluation.

Closes: Issues 4, 6, 9, 10

Before/after evidence: inference/policy_engine.py:276–288, 696–702 (NEEDS_REVIEW); inference/pipeline.py:305–326 (signal_health).

Phase F — Memory + Network retrain cluster (2026-09-10)

Both signals were retrained on a new 4-way split with a dedicated cal phase carved between test and train; the Platt calibrator was refit on cal (never seen by the booster fit or early stopping); operating thresholds were re-derived on cal alone; test was read exactly once, at the end, for reported numbers only.

Closes: Issues 2, 3 — for Memory and Network

Before/after evidence: scripts/split_memory.py, scripts/split_network.py (cal carve); scripts/train_memory.py:1–25, scripts/train_network.py:1–40 (split discipline docstrings).

Phase G — Static false-positive severity cluster (2026-09-18)

An NSRL SHA-256 allowlist and a real Authenticode chain-verification check were added ahead of Static's own ML judgment; a corroboration reason code was added when two or more capped signals agree; and Static's BLOCK verdict was allowed to escalate past the interim ALERT cap specifically when corroborated by a MALICIOUS memory or network verdict on the same scan.

Closes: Real-world false-positive risk (the same one Phase B raised)

Before/after evidence: features/authenticode_trust.py; features/nsrl_allowlist.py; inference/policy_engine.py:480–523, 664–679 (corroboration-gated escalation).

Phase H — Static retrain — this session (2026-09-21)

Cortex-Static received the same split-discipline treatment already proven on Memory and Network: a full retrain on regenerated (ExportsInfo-fixed) EMBER2024 data with a dedicated cal split, a memory-lifecycle fix that let the retrain complete without OOM-killing the process, candidate thresholds derived on cal with exact confidence intervals, a pre-registered acceptance gate, and a single authorized test read.

Closes: Issues 2, 3 — for Static

Before/after evidence: scripts/train_static.py; scripts/derive_static_thresholds.py; reports/static_retrain_20260921/.

6. Issue-by-Issue Resolution Audit

Each of the ten original issues below was checked directly against the current code, not against OPEN_ITEMS.md's own "DONE" labels. Where a fix is real but incomplete, it is marked PARTIALLY FIXED rather than rounded up.

Anchor-B Issue Resolution Scorecard (10 original EDR issues)		
1	Platt scaling fit on sigmoided probability, not raw margin models/static_lgbm.py, memory_lgbm.py, network_lgbm.py: calibrate() fits on raw_score=True booster margins.	FIXED
2	Same val split used for early-stopping AND calibrator fit Dedicated cal split (train/val/cal/test) for static (2026-09-21), memory & network (2026-09-10); cal excluded from booster fit + early stop.	FIXED
3	Thresholds chosen on test, then same test reused for metrics Thresholds now derived on cal via find_threshold_for_fpr(); test read exactly once, pre-registered acceptance gates (static).	FIXED
4	Raw AUC hides real-world FPR/PPV under low prevalence scripts/evaluate_all_models.py projects PPV / alert-rate / FPR at 1-in-1k/10k/100k prevalence for every signal.	FIXED
5	Behavioral CNN dataset-shortcut risks (sentinel tokens, no padding mask, train/serve short-trace mismatch) Padding mask + masked pooling FIXED (uncommitted in working tree); short-trace train/serve mismatch FIXED (item 5, Option B). Mal-AP-2019-only Cuckoo sentinel tokens (__anomaly__/_exception__) are NEVER filtered anywhere — untracked, undocumented.	PARTIALLY FIXED
6	Feature-extraction failures silently zero-filled feature_vector_with_report() returns degraded_groups; critical-group degradation routes the scan to NEEDS_REVIEW, not a trusted score.	FIXED
7	No automated parity test between training and inference feature extractors tests/test_static_feature_parity.py MVP covers vector contract, determinism, ExportsInfo, adapter passthrough. Real EMBER2024 record-schema check and thremler skew-quantification are explicitly still blocked (signify version conflict).	PARTIALLY FIXED
8	ExportsInfo always reports a fixed 128 hashed-bin count features/pe_features.py ExportsInfo.process_raw_features() now prepends len(raw) — the real export count.	FIXED
9	Unrelated ERROR conditions all mapped directly to ALERT New FinalDecision.NEEDS_REVIEW state: 'cannot analyze' no longer reported as a malware finding.	FIXED
10	Unavailable memory model produced ERROR→ALERT (health reported as malware evidence) ScanResult.signal_health is separate from the verdict; NOT_PROVIDED is neutral, only a real runtime model_error routes to NEEDS_REVIEW.	FIXED
FIXED PARTIALLY FIXED NOT ADDRESSED (none in this set) NOT APPLICABLE (none in this set)		

Figure 3 — issue resolution scorecard, color-coded by verified status.

#	Issue	Status	Evidence (file:line)
1	Platt scaling fit on the sigmoided probability instead of the raw booster margin	FIXED	models/static_lgbm.py:63–74 (PlattCalibrator), :108–109 (raw_score=True at inference), :297–324 (calibrate()); identically in models/memory_lgbm.py:64–82, 113–118 and models/network_lgbm.py:64–80, 111–116.
2	The same validation split was used for early-stopping AND for fitting the calibrator	FIXED	All three tree models now use a dedicated cal split, verified held out of both the booster fit and early stopping: scripts/train_static.py:26–41 (cal loaded in a separate pass, after training), scripts/train_memory.py:1–25, scripts/train_network.py:1–18.
3	Decision thresholds were chosen by searching the TEST set, then the same test set reused for metrics	FIXED	config/thresholds.yaml's static/memory/network sections document thresholds derived via find_threshold_for_fpr() on cal alone; scripts/derive_static_thresholds.py never opens the test parquet; reports/static_retrain_20260921/PREREGISTRATION.txt records the acceptance gate written down BEFORE the one-time test read.
4	Training data was balanced, hiding the real false-positive burden at realistic deployment prevalence	FIXED	scripts/evaluate_all_models.py:220–364 (PrevalenceProjection framework: PPV, alert-rate, expected FP/TP at 1-in-1,000 / 10,000 / 100,000), applied per signal at :467–757.
5	Behavioral CNN dataset-shortcut risks: dataset-only sentinel tokens, no padding mask, train/serve short-trace mismatch	PARTIALLY FIXED	Padding mask + masked pooling FIXED — models/behavioral_cnn.py:37–57 (key_padding_mask in attention), :118–136 (masked mean pool), regression-tested in tests/test_behavioral_model.py. Short-trace train/serve mismatch FIXED — tokenizer/api_tokenizer.py:24–30, 74–102 (commit 7a8a944, "item 5, Option B"). Dataset-only sentinel tokens NOT ADDRESSED: data/download_behavioral.py:45 documents that Mal-API-2019 (a malicious-only source) contains Cuckoo Sandbox markers "__anomaly__"/"__exception__" absent from the benign-containing sources, but no filtering of these tokens exists anywhere in the pipeline, and this specific risk is not mentioned in OPEN_ITEMS.md, README.md, or ARCHITECTURE.md at all.
6	Feature-extraction failures were silently replaced with zero-filled feature vectors	FIXED	features/pe_features.py:781–823 (raw_features records each failed group in degraded_groups instead of swallowing it), :825–853 (process_raw_features does the same for vectorization failures), :858–866 (feature_vector_with_report() returns the list). inference/pipeline.py:278–296 routes a CRITICAL degraded group to NEEDS_REVIEW rather than trusting the score.
7	Training used the published EMBER2024 vectors while inference used a separately-implemented raw extractor, with no automated parity test	PARTIALLY FIXED	tests/test_static_feature_parity.py is a real, passing MVP (vector contract, determinism, signed-binary authenticcode, the ExportsInfo count-slot regression guard, and byte-identical adapter passthrough between the live-PE and EMBER2024-record code paths). Its own module docstring states two sub-items remain: a real EMBER2024 record-schema check, and thrember skew-quantification — both explicitly BLOCKED by a dependency conflict (thrember needs pre-0.9 signify; this repo pins signify>=0.9,<0.10 for the opposite reason, item 4's fix).

#	Issue	Status	Evidence (file:line)
8	The ExportsInfo feature always reported a hashed-bin COUNT fixed at 128 instead of the real export count	FIXED	features/pe_features.py:585–599: ExportsInfo.process_raw_features() now prepends len(raw) — the real number of exported symbols — ahead of the 128-dim hash. Regression-tested in tests/test_static_feature_parity.py (test_exports_slot_is_count_not_constant, test_exports_slot_matches_raw_count).
9	Many unrelated ERROR conditions were all mapped directly to ALERT, conflating "could not analyze" with "found something suspicious"	FIXED	inference/policy_engine.py:276–284 (new FinalDecision.NEEDS_REVIEW state), :696–702 (decide()'s final rung: an ERROR-only outcome with no positive evidence returns NEEDS_REVIEW, never ALERT).
10	An unavailable/unconfigured memory model produced an ERROR that the policy converted to ALERT	FIXED	inference/policy_engine.py:346–360 (ScanResult.signal_health, explicitly separate from the security verdict); inference/pipeline.py:305–326 (_record_signal_health: a model that was never configured yields NOT_PROVIDED, which is neutral in decide() — only a configured model that actually raises at runtime produces ERROR, which then routes to NEEDS_REVIEW, not ALERT, the same as any other analysis failure).

For issue 5, "fully fixed" would require: either filtering the Cuckoo-sandbox sentinel tokens out of the vocabulary before training, or adding a held-out ablation that removes them and confirms the model's malicious-vs-benign separation does not depend on their presence.

For issue 7, "fully fixed" would require: the two blocked sub-items actually completed — a real EMBER2024 record pulled from HuggingFace and checked against the adapter's assumed schema, and a side-by-side comparison against the real thrember reference extractor run in a separate venv (since thrember and this repo's signify pin cannot coexist in one environment) to quantify the documented "skews toward malicious" gap.

7. Per-Signal Current State

Figure 1 in Part 3 summarizes authority and split scheme per signal. This section gives the full current picture for each, with an old-vs-new comparison against the numbers recorded in the prior audit report (compiled at commit 9331159) wherever a real number exists on both sides.

7.1 Cortex-Static

Looks at: the raw bytes of a Windows PE file — a 2568-dimension EMBER2024-compatible vector (byte histograms, header fields, imports/exports, section stats, Authenticode summary, string/IOC counts). **Architecture:** LightGBM, num_leaves=255, max_depth=15, learning_rate=0.03, min_child_samples=50, is_unbalance=True, 3000 boosting rounds with early stopping (models/static_lgbm.py:31–58). **Split scheme:** train / val / cal / test — a full split-discipline retrain completed this session (2026-09-21) on the regenerated (ExportsInfo-fixed) EMBER2024 parquet; cal is carved from the same seeded permutation as val, immediately after it, and is loaded in a separate pass after training to avoid a repeat of the OOM crash documented in Part 8. **Thresholds:** allow_below=0.4789517595186417, block_at_or_above=0.9811748406902472, both derived on cal via find_threshold_for_fpr() with exact Clopper–Pearson confidence intervals, then read against test exactly once under a pre-registered acceptance gate. **Authority:** BLOCK, but interim-capped to ALERT in decide() — escalates to a real BLOCK only when corroborated by a MALICIOUS memory or network verdict on the same scan (inference/policy_engine.py:432–523).

Metric	OLD (commit 9331159)	CURRENT (2026-09-21 retrain)
Test AUC-ROC	0.9988	0.998778 (regression gate ≥ 0.9985 : PASS)
ALLOW-boundary test FPR / detection	0.9999% / 98.03%	1.2129% / 98.21%
BLOCK-boundary test FPR / detection	0.0997% / 91.68%	0.1011% / 91.49% (95% CI touches the 0.1% bar)
Threshold derivation	find_threshold_for_fpr() directly against the TEST split (thresholds' recorded FPRs matched targets almost exactly — true by construction)	find_threshold_for_fpr() on cal; test read exactly once, after pre-registration
Dataset	2,340,000 train / 539,940 test, no cal split	Same row counts; regenerated with the ExportsInfo fix; cal split added
BLOCK authority	Uncapped, autonomous	Interim-capped to ALERT; corroboration-gated escalation only

The two AUC and detection numbers are close between old and new because the underlying booster hyperparameters are unchanged and EMBER2024 is a large, well-behaved dataset — the change that matters is not the headline number but how honestly it was obtained: the old ALLOW/BLOCK FPRs matched their targets almost exactly because they were derived directly against the same test split they were then scored on (issue 3, by construction); the new ones were derived on cal and show real in-sample-vs-out-of-sample drift when read against test for the first time, which is the expected signature of an honest derivation, not a regression.

7.2 Cortex-Behavioral

Looks at: a supplied ordered sequence of up to 100 Windows API-call names. **Architecture:** 1D-CNN (4 conv blocks, channel widths 128→256→128→128) + 4-head self-attention, token embedding dimension 128 by default (scripts/train_behavioral.py's CLI default; models/behavioral_cnn.py's own TrainConfig dataclass default is 64, used by Cortex-Emulation's copy of this architecture instead), dropout 0.3, sequence length 100 (models/behavioral_cnn.py:71–97). Unlike Emulation's copy of this same architecture, Behavioral's attention and pooling steps now use an explicit padding mask (key_padding_mask) so <PAD> positions are excluded from both

the attention computation and the pooled representation — this fix is present and regression-tested (tests/test_behavioral_model.py) but currently exists only in the working tree, uncommitted (git status shows models/behavioral_cnn.py as modified, not yet committed). **Split scheme:** train / val / test only — no cal split; the threshold is a manual sweep, not find_threshold_for_fpr(). **Threshold:** 0.60, chosen from a sweep over val+test combined (1,835 rows, 274 benign) rather than a precise per-mille FPR target, because 274 benign samples cannot support one. **Authority:** TERMINATE, uncapped — the highest authority in the system.

Metric	OLD (commit 9331159)	CURRENT
Test AUC-ROC / accuracy / precision / recall	0.9973 / 98.36% / 100% / 98.08%	0.997324 / — / 100% / 98.08% (all-rows test total, EVAL_ALL_MODELS_RESULTS.txt)
Threshold derivation	Sweep on val+test combined, 1,835 rows	Unchanged: same sweep, same 0.60
Known hard false positive	One persistent FP at score 0.921006 (networking-syscall benign trace), documented as unfixed	Same FP, same score, still documented as unfixed
Padding mask / short-trace scoring	Not present at 9331159	Both present in current code (padding mask uncommitted); short-trace floor at 10 calls, PENDING below it

The held-out numbers above are essentially unchanged from the old baseline because no retrain has happened for this signal; what changed is the code path those numbers were measured against. One caution: the EVAL_ALL_MODELS_RESULTS.txt file is explicitly dated "2026-09-08 10:31:09Z run at git 9153956" for its behavioral section, which predates the uncommitted padding-mask fix currently in the working tree — so the recall/precision numbers on file have not yet been re-measured against the masked-pooling model, only against unit tests that confirm the masking mechanism itself is correct.

7.3 Cortex-Memory

Looks at: a supplied 62-dimension feature vector (55 raw VolMemLyzer memory-forensics columns plus 7 hand-engineered ratio features — features/memory_features.py:55–112) derived from CIC-MalMem-2022 memory dumps. **Architecture:** LightGBM, num_leaves=31, learning_rate=0.05, 1000-round budget with early stopping, is_unbalance deliberately omitted (the train split is ~50/50) (models/memory_lgbm.py:37–61). **Split scheme:** train / val / cal / test, retrained 2026-09-10 with --cal-frac 0.2 (34,949 / 5,930 / 11,787 / 5,930 rows; val and test are byte-identical to the prior 3-way split). **Threshold:** 0.0024964628, derived on cal at target FPR 1% (56/5,860 benign FP, detection 99.93%). **Authority:** ALERT, capped — because 22 of its 62 features individually separate the classes above 0.95 AUC, consistent with CIC-MalMem-2022's entire benign class coming from repeated captures of one baseline Windows 10 VM; the retrain fixes calibration and threshold honesty, not this feature-fidelity concern, so the cap stays.

Metric	OLD (commit 9331159)	CURRENT (2026-09-10 retrain)
Test AUC-ROC / precision / recall / FPR	1.0 / 98.91% / 100% / 1.13%	0.999988 / 99.07% / 99.97% / 0.96% (EVAL_ALL_MODELS_RESULTS.txt)
Threshold	0.0005358335957155212 (target FPR 1% on val+test combined, 11,860 rows)	0.0024964628 (target FPR 1% on cal alone, 11,787 rows)
Split scheme	train / val / test, no cal	train / val / cal / test
Known limitation	22/62 features individually exceed 0.95 AUC — single-baseline-VM benign-collection artifact	Same finding, unchanged; explicitly why the ALERT cap stays

The honest recall at the new threshold (99.97%) is very slightly below the old val+test-derived claim of 100% — config/thresholds.yaml's own comment identifies that ~0.1 percentage-point gap as exactly the calibration/threshold optimism the split-discipline fix was meant to remove, not a regression.

7.4 Cortex-Network

Looks at: a supplied 78-dimension CSE-CIC-IDS2018 network-flow feature vector (Flow ID, source/destination IP and port, and timestamp are deliberately excluded from the feature columns). **Architecture:** LightGBM, num_leaves=31, learning_rate=0.05, is_unbalance=True (the real train split is ~84% benign / 16% malicious — models/network_lgbm.py:32–61). **Split scheme:** train / val / cal / test, retrained 2026-09-10 with --cal-frac 0.1 (1,331,042 / 211,697 / 195,715 / 213,217 rows). **Threshold:** 0.6672636218, derived on cal at target FPR 0.1% (159/160,863 benign FP, detection 96.26%). **Authority:** ALERT, capped, for two independent reasons: an Infiltration-class detection ceiling (~10–11% across both the old and new derivations, tied to CICFlowMeter's flow-statistics representation rather than under-training), and an externally-documented pattern of "nearly perfect" CSE-CIC-IDS2018 models degrading toward random performance on real traffic outside the dataset.

Metric	OLD (commit 9331159)	CURRENT (2026-09-10 retrain)
Test AUC-ROC / precision / recall / FPR	0.9972 / 99.69% / 97.52% / 0.098%	0.997182 / 99.70% / 97.50% / 0.09% (EVAL_ALL_MODELS_RESULTS.txt)
Threshold	0.9441855970306654 (target FPR 0.1% on val+test combined, 424,914 rows)	0.6672636218 (target FPR 0.1% on cal alone, 195,715 rows)
Infiltration-class detection	10.75%	10.19% (1433 test rows; per-attack-type table in EVAL_ALL_MODELS_RESULTS.txt)
Pipeline wiring	NOT wired into scan() — no network parameter existed	Wired via commit a8e6f45; inference/pipeline.py:159–164, 369–386

The near-identical Infiltration-class ceiling (10.75% old vs 10.19% new) across two entirely separate training runs is itself evidence this is a real representation limit of the dataset, not a training artifact.

7.5 Cortex-Emulation

Looks at: a supplied Speakeasy-emulation trace, restricted to module_entry entry points, of up to 500 API-name tokens. **Architecture:** the same 1D-CNN + 4-head self-attention shape as Behavioral, deliberately NOT copying Behavioral's padding-mask fix — models/emulation_cnn.py:78–87 and :148–157 still use plain, unmasked attention and a plain mean(dim=1) pool over all 500 positions, even though short sequences are common here too. embed_dim defaults to 64 (half of Behavioral's 128-default) because Emulation's real vocabulary is roughly 8× larger on a comparable-or-smaller training set. **Split scheme:** train / val / test only, preserving the dataset authors' own Jan-2022-train / Apr-2022-test collection-date split specifically to expose concept drift; no cal split, and none was added in this correction effort. **Threshold:** 0.999358594417572, for telemetry only — the target-FPR-1% point from a val+test sweep. **Authority:** report-only. Cortex-Emulation has no branch in decide() at all — grep the function and there is no emulation_verdict == line anywhere in it.

Metric	OLD (commit 9331159)	CURRENT
Test AUC-ROC	0.8737	0.873724 (EVAL_ALL_MODELS_RESULTS.txt — unchanged, no retrain occurred)
Test recall at telemetry threshold	~49.9% pooled	41.50% (test-era; matches the documented Jan→Apr collapse)
Concept-drift finding	Jan→Apr malicious recall 70.7%→41.5% at embed_dim=64, confirmed unfixed at embed_dim=32 (68.0%→38.3%)	Same finding; no new ablation run in this correction effort
Policy wiring	Report-only, no branch in decide() at all	Unchanged — still no branch in decide()

Emulation's numbers are unchanged because, unlike the other four signals, no retrain or correction pass has touched it since the old baseline — its report-only status and the padding-mask gap relative to Behavioral are both still-open items, carried into Part 10.

8. This Session's Specific Work: The Static Retrain

This is one signal's chapter in a five-signal report; the earlier report on this same retrain (docs/PhantomCortex_Static_Retrain_Report.pdf) covers the crash investigation, the fix, training, threshold derivation, the test read, and promotion at full length. Condensed here:

This Session's Static Retrain — 2026-09-21

Crash	Attempt 3 OOM-killed at ~36.6GB RSS lgb.Dataset() didn't inherit the n_jobs thread cap; cal's ~2.2GB array sat resident through the riskiest window unused.
Fix	train_from_holder() + holder-dict popping Caller builds a plain dict, never binds arrays to its own name; this function pops+deletes mid-call — verified via weakref+RssAnon, not assumed. cal loaded in a SEPARATE pass, after training.
Train	Booster retrained on regenerated EMBER2024 Exportsinfo-fixed train split (2,340,000 rows), 3000 rounds, num_leaves=255, is_unbalance=True. train/val carved from one seeded permutation; cal carved right after val.
Calibrate	Platt calibrator fit on cal, not val calibrate(model, X_cal, y_cal) — cal held out of BOTH the booster fit AND early stopping (fixes issues 1 + 2).
Thresholds	derive_static_thresholds.py sweep on cal Clopper-Pearson exact 95% CIs; menu only, no threshold chosen yet. allow_below @ target FPR 1%; block_at_or_above @ target FPR 0.05%.
Pre-register	PREREGISTRATION.txt — before test is ever read Chosen thresholds + acceptance gate (test AUC ≥ 0.9985) written down BEFORE the one-time test read. Addendum tightens the gate.
Test (once)	The one authorized held-out read test AUC 0.998778 (PASS ≥ 0.9985), allow_below FPR 1.21%, detection 98.21%, block_at_or_above FPR 0.10% (CI touches the architecture's 0.1% bar — AT it; not demonstrably below).
Promote	config/thresholds.yaml updated allow_below=0.478952, block_at_or_above=0.981175. Static BLOCK stays capped to ALERT by the interim policy — UNCHANGED by this retrain; only corroborated cases reach BLOCK.

Figure 5 — this session's static retrain, crash to promotion.

An earlier training attempt ("Attempt 3") was OOM-killed at roughly 36.6 GB RSS. The root cause was two-fold: `lgb.Dataset()` did not inherit the `n_jobs` thread cap used elsewhere in the training config, and the held-out `cal` split's ~2.2 GB array sat resident in memory through the single riskiest window of the run — Dataset construction plus boosting — without being used at all during that window. The fix, `models/static_lgbm.py::train_from_holder()`, restructures the caller into building a plain dict of arrays that is never separately named, so the training function can pop each array out and delete it the moment it is no longer needed; this was verified directly with `weakref` and `RssAnon` memory tracing, not assumed to work from reading the code. The `cal` split itself is loaded in a completely

separate pass, after training and calibration-object construction are already done.

With that fixed, the booster was retrained on the regenerated EMBER2024 train split (2,340,000 rows, now reflecting the ExportsInfo count fix from Part 6, issue 8) using the same hyperparameters as before (3000 rounds, num_leaves=255, is_unbalance=True), with train and val carved from one seeded permutation and cal carved immediately after val from the same permutation. The Platt calibrator was then fit on cal, not val. Candidate ALLOW and BLOCK thresholds were derived on cal alone via scripts/derive_static_thresholds.py, which reports exact Clopper–Pearson 95% confidence intervals rather than point estimates alone. Before test was ever opened, the chosen thresholds and a hard acceptance gate (test AUC ≥ 0.9985) were written down in reports/static_retrain_20260921/PREREGISTRATION.txt. The test split was then read exactly once: AUC 0.998778, passing the gate; ALLOW-boundary FPR 1.21% at 98.21% detection; BLOCK-boundary FPR 0.10% at 91.49% detection, with a confidence interval that touches the architecture document's 0.1% bar rather than clearing it. The new thresholds were then promoted into config/thresholds.yaml. Static's BLOCK verdict remains interim-capped to ALERT by the unrelated corroboration-gated policy from Phase G — this retrain does not touch that cap.

The five-file real-world check: a four-point trajectory

The prior audit's own 5-file validation, cortex-ml's 9331159-era re-run of it, and this session's own before/after re-check together give a fuller picture than any single comparison. All five files are confirmed-benign Windows binaries; cortex-endpoint's scores use the thrember reference feature extractor, while every cortex-ml column uses this repo's own features/pe_features.py.

File	cortex-endpoint (thrember)	cortex-ml @ 9331159 (pre-fix baseline)	cortex-ml pre-session (post Sept 8/10/18 fixes)	cortex-ml current (this session's retrain)	Current final decision
svchost.exe (signed MS)	0.4744 / allow	0.9839 / ALERT	0.877460 / ALERT	0.942081 / ALERT	ALERT
notepad_test.exe	0.0057 / allow	0.0062 / ALLOW	0.005642 / ALLOW	0.023364 / ALLOW	ALLOW
benign_test_50mb. exe	0.9690 / alert	0.9950 / BLOCK	0.980467 / BLOCK	0.984642 / BLOCK	ALERT (interim-capped)
notepad.exe	0.7746 / alert	0.9917 / ALERT	0.935731 / ALERT	0.960786 / ALERT	ALERT
extractor.exe (PyInstaller)	0.9820 / block	0.9956 / block	0.989866 / BLOCK	0.993514 / BLOCK	ALERT (interim-capped)

Every cortex-ml score for these five benign files has risen at every step since the original baseline, and two of them (benign_test_50mb.exe, extractor.exe) have sat at or above the BLOCK threshold since before this session's retrain even began. The final-decision column is what actually matters operationally: because the interim cap and corroboration-gating (Part 5, Phase B and G) were already in place before this session started, neither file reaches a real BLOCK outcome on a static-only scan — both demote to ALERT. The cause of the uniform upward drift across all five files was not isolated by an ablation in this repo; it is inferred to be some combination of a different booster and a different calibrator between checkpoints, not confirmed. What is confirmed, directly from the code, is that the known pe_features.py-vs-thrember feature-fidelity gap (issue 7, still open) remains the leading suspect, and that this is exactly why the interim BLOCK cap from Phase B has not been lifted by this retrain.

9. What's Better Now, System-Wide

- **Confidence numbers mean what they claim to mean.** All three tree-based signals' calibrators are now fit on raw margins on a split that never touched training or early stopping — the calibration-saturation failure mode that collapsed real scores into a narrow band is gone.
- **Thresholds were earned honestly.** Static, Memory, and Network now derive their operating points on a dedicated cal split, with test read at most once and, for Static, under a pre-registered acceptance gate written down before that read.
- **The false-alarm burden is visible.** Every signal's evaluation now projects PPV, alert rate, and expected false-positive count at realistic deployment prevalence (1-in-1,000 through 1-in-100,000), not just a balanced-data AUC that hides how many alerts a real deployment would actually generate.
- **"Could not analyze" is no longer reported as "found malware."** The new NEEDS_REVIEW state and the separate signal_health field mean a missing file, an oversized file, a non-PE file, or an unconfigured model routes to a review queue instead of the malware-alert stream.
- **Feature-extraction failures are visible, not fabricated.** A failed feature group is now recorded by name instead of silently becoming a zero vector that can itself look like evidence of maliciousness; a critical failure routes the whole scan to review rather than trusting a corrupted score.
- **Cortex-Network actually participates in decisions.** It went from a trained model with no wiring at all into the scan pipeline, to a fully wired, independently-running signal with its own retrained, cal-derived threshold.
- **Static's most dangerous authority is now checked.** The one previously-uncapped, autonomous verdict in the entire system (Static BLOCK) is now capped, escalating to a real block only when a second, independent signal agrees — closing the gap that let a single confidently-wrong static score autonomously block a legitimate file.
- **A known-file allowlist now exists.** A file with a matching NSRL hash or a real, chain-verified Authenticode signature is short-circuited straight to ALLOW for static's own judgment, without weakening any of the other four signals' independent scrutiny of the same file.
- **All four of these corrections are demonstrated, not asserted.** Every fix above is backed by a regression test, a pre/post measurement, or both — a feature-parity MVP, a policy-engine truth table, tensor-level padding-mask tests, and a Clopper–Pearson-bounded threshold-sweep script are all part of the current repository, not just claimed in a document.

10. What's Still Open, System-Wide

From the issue audit (Part 6)

- **Issue 5, residual:** the Cuckoo-sandbox sentinel tokens ("__anomaly__", "__exception__") present only in the malicious-only Mal-API-2019 source are never filtered from the behavioral vocabulary anywhere in the pipeline, and this specific risk is not tracked in OPEN_ITEMS.md, README.md, or ARCHITECTURE.md.
- **Issue 7, residual:** the feature-parity harness does not yet cover a real EMBER2024 record pulled from HuggingFace, and does not yet cover a side-by-side comparison against the real thrember reference extractor — both explicitly blocked by the signify version conflict between this repo (signify >=0.9) and thrember (needs pre-0.9 signify).

From the static retrain (Part 8)

- The BLOCK threshold's held-out test FPR confidence interval touches the architecture document's 0.1% bar rather than clearing it — it is AT the bar on real held-out data, not demonstrably below it.
- The real-world false-positive pattern is unresolved: two of the five confirmed-benign validation files still score at or above the BLOCK threshold, and every one of the five has drifted upward since the original baseline for a reason that was not isolated by any ablation in this repo.
- The interim BLOCK-authority cap remains in force, by design, until the feature-fidelity gap against thrember (issue 7's blocked residual) is actually closed — not lifted by this retrain, and not meant to be lifted until all three removal criteria in `inference/policy_engine.py:458–467` are met.
- A subsample setting referenced in the earlier static work remains inactive/unused in the current training configuration — not wired into any active code path.

System-wide, not tied to a single issue number

- **Cortex-Emulation's padding-mask gap.** The same short-sequence dilution problem that was fixed for Behavioral (`key_padding_mask` in attention, masked mean pooling) is still present, unfixed, in Emulation's copy of the same architecture (`models/emulation_cnn.py:78–87, 148–157`) — lower urgency only because Emulation is report-only and never reaches `decide()`.
- **Cortex-Emulation's concept-drift collapse is unresolved.** Malicious recall falls from 70.7% to 41.5% between the Jan-2022 and Apr-2022 collection eras, confirmed unfixed by simply reducing model capacity; only a materially different architecture or broader training data across more collection dates is scoped as the real fix, and neither has been started.
- **Memory and Network's ALERT caps are explicitly provisional, not permanent** — both still need real-world validation evidence beyond their respective lab-collected datasets (real injected-process samples for Memory; real network traffic outside CSE-CIC-IDS2018 for Network) before either cap could reasonably be raised.
- **The working tree currently holds large, uncommitted, and materially stale documentation.** `README.md` and `PROJECT_HISTORY_REPORT.md` both carry uncommitted local edits (confirmed via git status and git diff) that reinsert an old, 2026-08-27/28-dated narrative describing the system BEFORE almost every fix in this report — including describing the Platt-on-probability calibration bug as still open, describing no feature-parity test as existing, and describing Cortex-Static as "production-readiness DOWNGRADED." None of this reflects the current code. Anyone reading these two files directly, rather than the current `.py` files or the committed `OPEN_ITEMS.md`, would form a badly outdated picture of the system.
- **The behavioral padding-mask fix itself is uncommitted.** It is real, present in the working tree, and regression-tested (`tests/test_behavioral_model.py`, also uncommitted) — but it does not yet exist in any git commit, so it is invisible to `git log` and was found only by reading the actual current file contents, as this report was instructed to do.

11. What Changed, Concretely

The 40 commits between 9331159 and HEAD, oldest first, grouped by the phase each belongs to (Part 5 has the full narrative for each phase).

Phase A

- 6eed759 Cortex tree models: fit Platt calibrators on raw margins, not probabilities
- 453a495 gitignore: ignore *.zip and stray scp "user@host" local files
- 6e8d50c Cortex tree signals: re-derive thresholds for the raw-margin calibrators

Phase B

- b11999f policy_engine: interim-cap Cortex-Static BLOCK to ALERT (item 2)
- 961f4a3 pipeline: run behavioral on static BLOCK too (item 2 companion)

Phase C

- bb1ade0 pe_features: ExportsInfo prepends export COUNT, not constant 128
- 6db8cdb signify: pin $\geq 0.9, < 0.10$, port to 0.9.x API, fail hard if missing (item 4)
- 7a8a944 behavioral: score short (10-99 call) traces, floor PENDING at < 10 (item 5, Option B)
- dc55cdc Track Architecture.txt (was untracked); fold in item 2 + item 5 corrections

Phase D

- a8e6f45 pipeline: wire Cortex-Network into scan() via pre-supplied features (item 6)
- 57785fc policy_engine: load thresholds from config/thresholds.yaml (single source of truth)
- 5044b86 Add OPEN_ITEMS.md tracker
- c6ebe5c tests: feature-parity harness for pe_features.py (item 7 MVP)
- 9153956 tests: policy-engine truth table + ONNX-parity pytest (item 8)
- c033f27 chore: remove stray files and caches; gitignore .pytest_cache/
- bfd6657 docs: promote ARCHITECTURE.md as canonical, retire Architecture.txt
- 32cb00a docs: fold reason.txt's per-model architecture rationale into ARCHITECTURE.md
- 45dd489 eval: standalone all-models val+test harness at deployed thresholds

Phase E

- 6067d77 policy_engine: add NEEDS_REVIEW state; "cannot analyze" is not "malware" (item 9)
- 4a43439 pipeline: record model health on ScanResult.signal_health, separate from verdict (item 10)
- fb254aa pe_features: report feature-group failures instead of silent zero-fill (item 6)
- 39e4c02 evaluate_all_models: project every model onto realistic deployment prevalence (item 4)

Phase F

- a98c8c7 docs(OPEN_ITEMS): record agreed split-scheme design for the retrain cluster (items 2, 3, 8-residue)
- 7a7d906 split_memory, split_network: add a cal phase carved between test and train (retrain cluster, item b)

- 4490743 `train_memory,train_network`: fit the Platt calibrator on a held-out cal split, drop `--test` (retrain cluster, item c)
- 11668e6 `thresholds`: re-derive memory/network operating points on the cal split (retrain cluster, item d)
- 7dc7f47 `evaluate_all_models`: relocate network's per-attack-type breakdown; regenerate memory/network results (retrain cluster, item f)
- fcd4e61 `docs(OPEN_ITEMS)`: mark the memory+network retrain half done; static still open

Phase G

- 383edf1 `features/authenticcode_trust`: real Authenticode chain verification, standalone from the ML feature (static FP cluster, addition A)
- 3749989 `data/download_nsrl,features/nsrl_allowlist`: NSRL SHA-256 allowlist artifact + loader (static FP cluster, addition A)
- 3449503 `pipeline`: wire known-file allowlist into `scan()` step 3b (static FP cluster, addition A)
- b6efb22 `policy_engine`: add `corroborated_multi_signal` reason code to `decide()` (static FP cluster, addition B1)
- ac132d2 `policy_engine`: corroboration-gated static BLOCK escalation (static FP cluster, addition B2)
- fbdd207 `docs(OPEN_ITEMS)`: mark the static false-positive severity cluster fully done (addition A, B1, B2)
- 4f45630 `pipeline`: behavioral config-gap becomes `NOT_PROVIDED`, matching memory/network
- 388d6df `requirements`: pin scikit-learn and pefile upper bounds

Phase H

- 8192452 `static`: split-discipline retrain + memory-lifecycle fix (`train_from_holder`, calibrate on cal)
- 48f55f0 `scripts`: derive static ALLOW/BLOCK candidate thresholds on the cal split (Clopper-Pearson CIs)
- 9030d6a `docs(OPEN_ITEMS)`: record static split-discipline retrain, pre-registration, one-time test read and promotion; add `reports/static_retrain_20260921`
- 438d936 `config(thresholds)`: promote static split-discipline retrain (thresholds derived on cal, not test)

This session's four commits, in detail

388d6df — pinned scikit-learn to `>=1.4,<2.0` and pefile to `>=2023.2.7,<2025` in `requirements.txt`, closing a dependency-pinning gap left over from item 6 (a real, source-confirmed pefile behavior change — an unconditional `gc.collect()` added to `PE.close()` in 2024.8.26 — measured at $\sim 1.04\times$ overhead per scan on this repo's own fixture).

8192452 — the retrain itself: `models/static_lgbm.py::train_from_holder()` restructured to fix the OOM crash (Part 8), the booster retrained on regenerated EMBER2024 data, and the calibrator fit on the new cal split via `calibrate(model, X_cal, y_cal)`.

48f55f0 — `scripts/derive_static_thresholds.py` added: a read-only script that sweeps candidate ALLOW/BLOCK thresholds on cal alone, reporting exact Clopper–Pearson confidence intervals for each candidate, and explicitly never opens the test parquet (verified by the script's own docstring and its reliance on `scripts.train_static's` cal-loading function, never its test-loading equivalent, since no such equivalent exists in that module).

9030d6a and **438d936** — the promotion: `OPEN_ITEMS.md` updated with the pre-registration record, the one-time test read, and the promotion decision; `config/thresholds.yaml's` static section updated to the new `allow_below/block_at_or_above` values with a full derivation comment, superseding but not deleting the two prior threshold values.

12. What's Left

- **Commit the behavioral padding-mask fix and its tests.** `models/behavioral_cnn.py` and `tests/test_behavioral_model.py` are real, working, and uncommitted — the single highest-value, lowest-risk next action, since the fix already exists and is already tested.
- **Reconcile or discard the stale README.md / PROJECT_HISTORY_REPORT.md working-tree edits.** Both files currently contradict the committed, current state of the system; either finish the "dedicated docs pass" repeatedly deferred across commit messages, or revert the stale hunks so the working tree matches what is actually true.
- **Close the sentinel-token risk in Cortex-Behavioral** (issue 5's residual) — filter the Mal-API-2019-only Cuckoo markers out of the vocabulary, or run an ablation proving the model does not depend on them.
- **Finish the feature-parity work** (issue 7's two blocked sub-items): pull a small real EMBER2024 record sample from HuggingFace to check the adapter's schema assumption, and run the thrember skew-quantification cross-check in a separate venv (thrember needs pre-0.9 signify, incompatible with this repo's own signify $\geq$ 0.9 pin) — this is also the single blocker standing between Static and ever regaining unilateral BLOCK authority.
- **Give Cortex-Emulation the same padding-mask fix already proven on Behavioral**, and either scope and start the concept-drift fix (engineered behavioral-category features, or training data spanning more collection dates) or make an explicit decision to leave Emulation report-only indefinitely.
- **Collect real-world validation data for Memory and Network** beyond their respective lab-collected datasets, so their ALERT caps can be revisited on evidence rather than left provisional indefinitely.
- **Investigate the uniform upward score drift** across this session's own before/after 5-file check — it was observed and reported but not isolated by an ablation; a controlled comparison (same booster hyperparameters, only the calibrator changed, vs. only the training data changed) would settle whether it is calibrator behavior, the ExportsInfo fix, or something else.
- **No owner is currently assigned** to any of the items above; they exist as findings in this report and in OPEN_ITEMS.md/README.md's open-items sections, not as an active work queue.

Closing. PhantomCortex started with ten real, documented correctness problems and has spent five separate work sessions across two weeks fixing them one deliberate phase at a time — not a single cleanup pass, but a sustained habit of finding a real bug, fixing it, proving the fix with a test or a measurement, and writing down exactly what still isn't fixed. Eight of the ten original issues are now genuinely closed, verified directly against the code rather than taken on faith from a tracker. The two that remain open are open honestly: a narrow, specific, and previously undocumented risk in how the behavioral model's vocabulary was built, and a feature-parity check that exists as a real, tested first step but deliberately does not yet claim more than it has actually verified. The system's most dangerous single authority — a static model's ability to autonomously block a file — is now checked by a second signal before it can act alone, and the evidence for keeping that check in place is current, not historical: two of five real benign test files still score above the block line today. What is asked of a reader who sees only this paragraph is this: the system is meaningfully more honest and more carefully checked than it was, real work remains before its most autonomous decision can be trusted unconditionally, and the gap between those two facts is now visible in the code rather than hidden by it.